

nopCommerce Demo Store Testing Plan

QA automation capstone scope document using Playwright with JavaScript.

Field	Details
Project type	E-commerce website functional testing capstone
Selected website	nopCommerce demo store
Website URL	https://demo.nopcommerce.com/
Automation stack	Playwright with JavaScript
Minimum requirement	8 services, 15 test cases per service, total 120 test cases
Testing focus	Frontend user flows, form validations, cart and checkout behavior, account-related pages, and reporting evidence

Project Overview

This capstone project will test the nopCommerce demo store from the point of view of a QA engineer. The project will not involve creating or modifying the website. The complete work will focus on understanding the website, preparing service-wise test cases, automating selected flows, executing the tests, and preparing evidence for submission.

The website has realistic e-commerce features such as registration, login, search, product categories, cart, wishlist, compare products, checkout, addresses, order history, and customer support. Because these features are connected to normal online shopping journeys, the site is suitable for a capstone project that requires at least 120 total test cases.

Reason For Website Selection

- It is a public demo e-commerce application and can be accessed without building a website locally.
- It contains enough modules to divide the capstone into 8 clear services.
- It supports realistic customer journeys such as browsing, cart management, checkout, account pages, and support pages.
- It can be tested using Playwright with JavaScript, matching the training technology stack.
- The site is reset regularly, so test data can be planned as disposable dummy data.

Project Objectives

- Prepare a complete test plan for the selected website.

- Create a minimum of 120 test cases across 8 services.
- Cover positive, negative, validation, navigation, and regression scenarios.
- Automate the planned tests using Playwright with JavaScript.
- Generate execution evidence through test reports, screenshots, and traces wherever required.
- Present the project as a QA/testing capstone, not as a development project.

Scope Of Testing

The scope is limited to the public frontend demo store. The project will focus on customer-facing functionality and avoid any activity that requires real payment, real personal information, or backend administration.

In Scope	Out Of Scope
Registration, login, logout, and account navigation	Real payment processing or real order fulfillment
Product category, product detail, search, sorting, and filtering behavior	Backend database validation or admin panel modifications
Cart, wishlist, compare products, and checkout UI flow	Performance testing, security testing, and load testing
Address, shipping, order confirmation, and order history pages	Testing with real customer data or saved real card details
Contact Us, newsletter, recently viewed, and navigation links	Changing product data, price data, or server configuration

Service-Wise Test Case Distribution

The capstone requirement will be met by preparing 15 test cases for each of the following 8 services. This gives a planned total of 120 test cases irrespective of browser selection.

No.	Service / Module	Testing coverage	Count
1	Registration and Authentication	Register, login, logout, invalid credentials, required fields, email format validation, duplicate email handling, and account session behavior.	15
2	Product Catalog	Home page products, category pages, product detail page, price display, images, availability, product attributes, and related navigation.	15
3	Search and Filters	Search box, search result accuracy, empty search, advanced search, category search, sorting, paging, and manufacturer/category filters.	15
4	Cart	Add to cart, update quantity, remove item, cart subtotal, terms checkbox, continue shopping, estimate shipping, and cart persistence.	15
5	Wishlist and Compare	Add to wishlist, remove wishlist item, update wishlist quantity, email wishlist, add to compare, remove compare item, and compare products table.	15
6	Checkout and Payment	Guest checkout, registered checkout, billing step, shipping step, payment method, payment information, order review, and order confirmation.	15
7	Address and Shipping	Billing address, shipping address, country/state selection, required	15

		address fields, shipping method, validation messages, and address display during checkout.	
8	Customer Support and Account	Contact Us form, newsletter, My Account page, order history, downloadable products, reward points, change password, and account navigation.	15

Detailed Module Coverage

Module	Detailed planned coverage
Registration and Authentication	Verify mandatory field messages, successful registration with dummy data, login with valid user, login failure with invalid password, logout behavior, and protected account page access.
Product Catalog	Verify category menu navigation, product tiles, product detail page information, product price, image visibility, attribute selection, stock/availability text, and add-to-cart availability.
Search and Filters	Verify keyword search, empty search, no-result search, advanced search, search within category, product sorting, page size, and navigation between result pages.
Cart	Verify adding simple and configurable products, product quantity changes, remove item, cart total, discount/gift card fields if available, terms checkbox behavior, and continue shopping flow.
Wishlist and Compare	Verify wishlist add/remove, quantity update, sharing/email option, compare list add/remove, clear compare list, and comparison details such as product name and price.
Checkout and Payment	Verify guest checkout and registered checkout, billing and shipping step validation, payment method selection, payment information, confirm order, and final order success page.
Address and Shipping	Verify address form fields, required validations, country and state dropdown behavior, saved address reuse, shipping method selection, and address shown in checkout summary.
Customer Support and Account	Verify Contact Us form validation, successful contact submission, account menu links, orders page, address page, change password page, newsletter subscription, and recently viewed products.

Types Of Testing Planned

Testing type	Purpose in capstone
Functional testing	Confirm that each feature works according to the expected user flow.
Validation testing	Check required fields, invalid inputs, empty forms, incorrect email formats, and form-level messages.
Negative testing	Verify that the application handles wrong inputs and incomplete actions correctly.
Navigation testing	Check links, menu navigation, breadcrumbs, page redirection, and account page movement.
Regression testing	Re-run automated cases after changes in scripts or test data to ensure existing flows still work.
UI assertion testing	Verify headings, visible messages, product names, cart counts, totals, and confirmation text.

Automation Approach

Automation will be created using Playwright with JavaScript. The tests will be organized service-wise so that each module can be maintained separately and executed independently if needed.

- Use Playwright Test as the test runner.
- Create separate spec files for major services such as authentication, catalog, search, cart, wishlist, checkout, address, and account/support.
- Use stable locators such as role, label, text, placeholder, and visible UI selectors wherever possible.
- Use assertions to verify page URLs, visible messages, product names, cart counts, table rows, form validations, and order confirmation text.
- Use hooks or reusable helper functions only where they reduce repeated setup, such as registration/login or adding products to cart.
- Keep test data separate from the test logic so dummy names, emails, addresses, and product names can be changed easily.

Planned Project Structure

Folder / File	Purpose
tests/authentication.spec.js	Registration, login, logout, and authentication validation test cases
tests/productCatalog.spec.js	Product listing, category, product detail, and product attribute test cases
tests/searchFilters.spec.js	Search, advanced search, sorting, filtering, and paging test cases
tests/cart.spec.js	Cart add, remove, update quantity, total, and cart navigation test cases

tests/wishlistCompare.spec.js	Wishlist and compare product test cases
tests/checkoutPayment.spec.js	Checkout journey, payment method, review order, and order confirmation test cases
tests/addressShipping.spec.js	Address form, shipping method, and address display test cases
tests/accountSupport.spec.js	Contact Us, account pages, orders, password, newsletter, and support-related test cases
test-data/	Reusable dummy data for users, addresses, products, and checkout forms
playwright-report/	HTML execution report generated after running the suite

Test Data Strategy

The website is a public demo site, so all test data will be dummy data. Unique email addresses will be generated for registration flows to avoid duplicate account issues.

- Use dummy user names, phone numbers, addresses, company names, and emails.
- Avoid real customer data, real card data, or real personal information.
- Use products already available on the demo store, such as computers, electronics, apparel, books, jewelry, and gift cards.
- Prepare reusable data objects for registration, billing address, shipping address, and contact form scenarios.
- Handle site reset behavior by creating fresh data when required.

Entry And Exit Criteria

Entry criteria	Exit criteria
Website is accessible and major pages are loading.	120 test cases are documented service-wise.
Testing scope and services are finalized.	Automation scripts are created for the planned scope.
Playwright with JavaScript setup is available.	Test execution report is generated and reviewed.
Dummy test data is prepared.	Defects or failed cases are noted with evidence.
Browser execution approach is agreed.	Final capstone files are organized for submission.

High-Level Test Scenarios

Service	Example high-level scenarios
Registration and Authentication	Verify successful registration, invalid registration, successful login, failed login, logout, and account access behavior.
Product Catalog	Verify that customers can browse categories, view product details, select product attributes, and see correct product information.
Search and Filters	Verify that search returns matching products, handles no-result searches, supports advanced options, and allows sorting and page size changes.
Cart	Verify that products can be added, quantity can be changed, products can be removed, and cart totals update correctly.
Wishlist and Compare	Verify that products can be added to wishlist and compare list, removed from both, and viewed in the expected list/table.
Checkout and Payment	Verify guest and registered checkout, required checkout steps, payment method selection, order review, and confirmation message.
Address and Shipping	Verify address field validation, country/state behavior, saved address selection, shipping method display, and checkout address summary.
Customer Support and Account	Verify contact form behavior, account navigation, order page, password change validations, newsletter subscription, and recently viewed products.

Defect Reporting Plan

If any test case fails during manual verification or automation execution, it will be recorded clearly so the issue can be understood without rerunning the full test immediately.

Defect field	Details to capture
Defect ID	Unique ID such as BUG-001
Module	Service where the issue was found
Summary	Short description of the problem
Steps to reproduce	Clear numbered steps from the test case
Expected result	What should have happened
Actual result	What actually happened
Evidence	Screenshot, trace, report link, or error message
Severity / priority	Impact and urgency of the issue
Status	Open, fixed, retest, or closed

Reporting And Evidence

- Playwright HTML report will be used as the main execution report.
- Screenshots will be captured for important failed scenarios or final evidence if required.
- Trace files can be used to debug failed tests and show step-by-step execution.
- Final summary will include total tests planned, total tests automated, pass count, fail count, and pending items.

Risks And Mitigation

Risk	Mitigation
Demo site resets regularly	Use fresh dummy data and avoid depending on old registered accounts.
Product names or prices may change	Use flexible assertions and update test data if visible website data changes.
Public site may be slow or temporarily unavailable	Use Playwright timeouts carefully and rerun after confirming site availability.
Some checkout paths may need specific product types	Select products that support predictable cart and checkout behavior.
Automation locators may break if UI changes	Prefer stable user-facing locators and keep selectors organized.

Assumptions

- The capstone will run against the public frontend demo store at <https://demo.nopcommerce.com/>.
- Only dummy test data will be used.
- The primary automation framework will be Playwright with JavaScript.
- The 120 test cases will be counted service-wise, not browser-wise.
- If the website resets or changes, affected test cases will be updated with trainer approval.

Planned Capstone Flow

1. Finalize website and service-wise testing scope.
2. Prepare 120 test cases across 8 services.
3. Review and improve test cases based on trainer feedback.
4. Create Playwright with JavaScript project structure.
5. Automate tests service by service.
6. Execute tests, review failures, and fix automation issues.
7. Prepare final report, screenshots/traces, and submission folder.

Final Deliverables

Deliverable	Description
Capstone testing plan	This document explaining website selection, scope, services, approach, and expected output.
Test case document	Service-wise list of 120 test cases with steps, expected results, priority, and status.
Playwright automation project	JavaScript test scripts organized by service/module.
Test data files	Dummy data required for registration, login, address, checkout, and forms.
Execution report	Playwright HTML report showing test run results.
Evidence folder	Screenshots, traces, or notes for important passed/failed scenarios.
Final summary	Brief project summary with coverage, execution result, known issues, and conclusion.

References

- nopCommerce demo store frontend: <https://demo.nopcommerce.com/>
- nopCommerce official demo information: <https://www.nopcommerce.com/en/demo>
- Playwright documentation will be used later during automation setup and script execution.

Conclusion

The selected website and planned scope are suitable for the capstone requirement because the application has enough independent services to prepare 120 test cases and enough realistic e-commerce behavior to demonstrate QA automation skills. The project will be completed using Playwright with JavaScript, matching the training stack and keeping the work focused on testing activities.